Subject Code	:	**CEUE203**	**Semester**	:	**3**	**Academic Year**	: 2026-27
Subject Name	:	Object-Oriented Programming (Java)					

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY (CHARUSAT)

Chandubhai S. Patel Institute of Technology (CSPIT) — Department of Computer Science & Engineering

MINI-PROJECT EVALUATION REPORT

MiniBank — Enterprise-Grade Object-Oriented Core Banking System

Candidate Name: **Arya Shah**	**Student ID / Roll No:** **25CE107**
Course / Branch: B.Tech Computer Engineering (Sem 3)	**Course Code / Title:** CEUE203: Object-Oriented Programming
GitHub Repository: **https://github.com/Aryashah1505/oop_java-banking_system-25ce107.git**	**Submission Date:** October 2026

1. EXECUTIVE SUMMARY & PROJECT OBJECTIVES

MiniBank is a production-grade, object-oriented console core banking application engineered in Java (JDK 17+). Designed as a comprehensive culmination of the CEUE203 curriculum, MiniBank bridges foundational academic OOP theory with modern software engineering paradigms, including clean architectural layering, relational database persistence (SQLite via JDBC), defensive security with cryptographic PBKDF2 hashing, resilient transactional integrity, and automated unit/integration test suites.

Core Objectives: Primary pedagogical and technical objectives accomplished:

- **Object-Oriented Mastery:** Strict adherence to encapsulation, abstract inheritance hierarchies, interface-driven contracts, polymorphism, and Java 16+ records/sealed classes.
- **ACID Transaction Safety:** Zero balance corruption during account-to-account funds transfers, deposit/withdrawal validation, and atomic rollbacks upon failure.
- **Dual-Tier Storage Architecture:** Seamless operation with relational SQLite persistence, automatic database schema bootstrapping, and automatic migration from legacy serialized binaries (.dat).
- **Security & Access Control:** Role-Based Access Control (RBAC) separating System Administrators from Customers, PBKDF2 password derivation with cryptographically secure per-user salt, and automated brute-force lockout mechanisms.
- **Comprehensive Test Automation:** 100% build verification achieved with 45 unit and integration tests executed under JUnit 5 and automated via GitHub Actions CI pipeline.

2. SYSTEM ARCHITECTURE & OOP CURRICULUM MAPPING

The MiniBank project strictly implements and expands upon all major object-oriented design topics instructed throughout the laboratory sessions:

Lab / Topic Module	Core OOP Concept Implemented	MiniBank Source Code Reference
Lab 1: Class Fundamentals	Enums, Java Records, Switch Expressions	`BankInfo (Record), MenuOption (Enum), MiniBank.java`
Lab 2: Encapsulation & Mutability	Private data hiding, invariants, class counters	`Customer.java, Account.java (Thread-safe atomic IDs)`
Lab 3: Object Contracts & Cloning	equals(), hashCode(), Deep Cloning, toString()	`Customer.clone(), Address.java deep copy, Account equality`
Lab 4: Validation & Formatting	Regex validation, StringBuilder, Command Records	`Validator.java (IFSC/Phone/Email), StatementFormatter.java`
Lab 5: Polymorphism & Inheritance	Abstract base classes, Template Method pattern	`Account.java (abstract), SavingsAccount, CurrentAccount, FD`
Lab 6: Interface Contracts	Multiple inheritance of type, Default & Functional	`Transactable, InterestBearing, WithdrawRule, Premium`
Lab 7: Custom Annotations & Reflection	Runtime custom annotations, metadata inspection	`AnnotationValidator.java, @Id, @Positive, @MaxLength`
Lab 8: Exceptions & Persistence	Custom exception hierarchy, SQLite JDBC, I/O	`BankException hierarchy, DatabaseManager, AccountStore`

3. DETAILED MODULE SPECIFICATIONS & BUSINESS RULES

---

1. Specialized Account Hierarchy: MiniBank implements a diverse account hierarchy derived from the abstract Account base class:

- **Savings Account (SavingsAccount.java):** Maintains strict minimum balance constraint (₹500.00). Accrues 4.0% p.a. compound interest computed with exact monetary precision using BigDecimal.
- **Current Account (CurrentAccount.java):** Designed for corporate and high-velocity commerce. Incorporates an approved overdraft limit of up to ₹10,000.00 beyond zero balance.
- **Fixed Deposit Account (FixedDepositAccount.java):** High-yield investment vehicle offering 7.0% p.a. guaranteed interest. Enforces strict withdrawal lockout rules before the contract maturity date.

2. Transaction Processing & Fault Invariance: All monetary movements are governed by robust architectural invariants:

- **Atomic Transfer Mechanism:** Transfers across accounts are executed under strict transactional isolation. If a destination rejects the credit or the source lacks cleared funds, the operation fails completely with zero balance alteration.
- **Defensive Validations:** Rejection of zero, negative amounts, self-transfers (fromAccount == toAccount), and transfers from locked accounts.
- **Configurable Velocity Limits:** Accounts enforce daily transaction caps (default ₹50,000/day) preventing velocity-based fraud.
- **Audit Logging:** Every transaction creates a permanent immutable record with timestamp, source, target, delta amount, ending balance, and execution status.

3. Cryptographic Security & RBAC: Industrial-strength authentication architecture protecting financial assets:

- **PBKDF2 with HMAC-SHA256:** Passphrases undergo 65,536 iterations of PBKDF2 key stretching combined with a cryptographically secure 16-byte random salt per user (zero plain text storage).

- **Brute-Force Protection:** Automatic account lockout triggered immediately after 5 consecutive failed authentication attempts.
- **Segregated Role Dashboards:** Role-Based Access Control dynamically confines Customers to their respective balances and statements, while empowering Administrators with account lifecycle controls, liquidity oversight, and audit trails.

4. RELATIONAL DATABASE SCHEMA (SQLITE JDBC)

The application transitions persistence from fragile file-based serialization to a production-grade relational database powered by SQLite via JDBC (minibank.db).

Table Name	Primary / Foreign Keys	Stored Attributes & Invariants
customers	PK: customer_id (INTEGER)	name, email (UNIQUE), phone, street, city, state, zip_code
accounts	PK: account_id (INTEGER), FK: customer_id	account_number (UNIQUE), account_type, balance, min_balance/ overdraft/maturity, is_locked, daily_limit
transactions	PK: transaction_id (INTEGER), FK: account_id	timestamp, type (DEPOSIT/ WITHDRAW/TRANSFER), amount, balance_after, status
users	PK: username (VARCHAR)	password_hash (PBKDF2), salt (HEX), role (ADMIN/CUSTOMER), FK: customer_id, failed_attempts, is_locked

5. TESTING STRATEGY & VERIFICATION EVIDENCE

MiniBank follows a test-driven development (TDD) ethos. The automated test suite encompasses 45 discrete unit and integration test assertions spanning all business logic scenarios, negative edge cases, and stress conditions.

Automated Test Suite Summary: Comprehensive Test Coverage Breakdown:

- **MiniBankTest.java (34 Tests):** Validates core domain objects, customer deep cloning, regex pattern validators, account deposit/withdraw rules, overdraft limits, interest accrual, custom reflection-based annotation validators, and CSV export functionality.
- **EnhancedBankingTest.java (11 Tests):** End-to-end integration tests verifying atomic multi-account transfers, velocity daily limit enforcement, administrative account locking/unlocking, and SQLite relational persistence consistency.

```
[INFO] -------------------------------------------------------
[INFO]  M I N I B A N K   A U T O M A T E D   T E S T   R U N
[INFO] -------------------------------------------------------
[INFO] Running EnhancedBankingTest
Deposited 1200.00 -> ACC002001. Balance: 6200.00
Withdrew 300.00 <- ACC002001. Balance: 5900.00
Account ACC002007 has been LOCKED.
Account ACC002007 has been UNLOCKED.
Transferred 1000.00 from ACC002015 -> ACC002016
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0 -- in EnhancedBankingTest
[INFO] Running MiniBankTest
Transferred 2000.00 from ACC002030 -> ACC002031
Interest applied: 400.00. New balance: 10400.00
[INFO] Tests run: 34, Failures: 0, Errors: 0, Skipped: 0 -- in MiniBankTest
[INFO]
[INFO] Results: Tests run: 45, Failures: 0, Errors: 0, Skipped: 0
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS (Total time: 1.391 s)
[INFO] ------------------------------------------------------------------------
```

6. CONCLUSION & SUBMISSION CHECKLIST

The MiniBank project satisfies and exceeds the requirements for the CEUE203 Object-Oriented Programming curriculum. It models a complete, enterprise-ready banking engine upholding rigorous standards of object encapsulation, relational data integrity, and software quality assurance.

Artifact / Requirement	File / Location	Status & Verification
Complete Project Source Archive	oop_java-banking_system-25ce107.zip	**Verified (2.5 MB) & Committed in Repository**
GitHub Remote Repository	Aryashah1505/oop_java-banking_system-25ce107	**Publicly Accessible & Branch 'main' Synchronized**
Automated CI Pipeline	.github/workflows/ci.yml	**Passing (Automated compilation and test execution)**
Viva Voice Exam Preparation Guide	VIVA.md	**Complete Q&A covering Labs 1 through 8**
Evaluation Document (Report)	25CE107_OOP_Java_MiniBank_Project_Report.docx / .pdf	**Formatted as per CHARUSAT CSPIT academic standards**

Submitted by:
Arya Shah (Roll No: 25CE107)
Department of Computer Science & Engineering
CSPIT, CHARUSAT University